Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

Report on laboratory work no. 5

Course: Operating Systems

Topic: The File System with Your Own Hands

Track A: warm-up on Linux

Done by:
st. gr. FAF-241,
Serghei Barladean

Checked by:
Patricia Reitman

Chișinău – 2026

Contents

Purpose of the Work....3
Part 1. The Tree and the File Types....3
1.1. Work Done....3
1.2. Observations....4
Part 2. Inodes and Links....5
2.1. Work Done....5
2.2. Observations....6
Part 3. Permissions....7
3.1. Work Done....7
3.2. Observations....8
Part 4. Disks and the Kernel as Files....9
4.1. Work Done....9
4.2. Observations....10
Part 5. Stretch Tasks....11
5.1. Magic Numbers....11
5.2. Deleting a File That Is Still Open....11
Conclusions....12

Purpose of the Work

The purpose of this laboratory work is to look at the Linux file system from the user side and to check the lecture material on a real machine: the file types in the directory tree, the inode that stays behind a name, hard and symbolic links, the permission bits, and the disks and kernel data that Linux shows as files. No program is written. Every statement in the report rests on the output of a command.

The work was done on Ubuntu 24.04 (aarch64) in a Multipass virtual machine on macOS, as the normal user `ubuntu` (the command `whoami` printed `ubuntu`, not `root`), in the folder `~/os-lab5`. The figures are taken from the recorded terminal session.

Part 1. The Tree and the File Types

1.1. Work Done

The first group of commands lists the root directory and shows the type of a few typical paths:

```
ls /
ls -ld /etc /etc/passwd /bin /dev/null /tmp
```

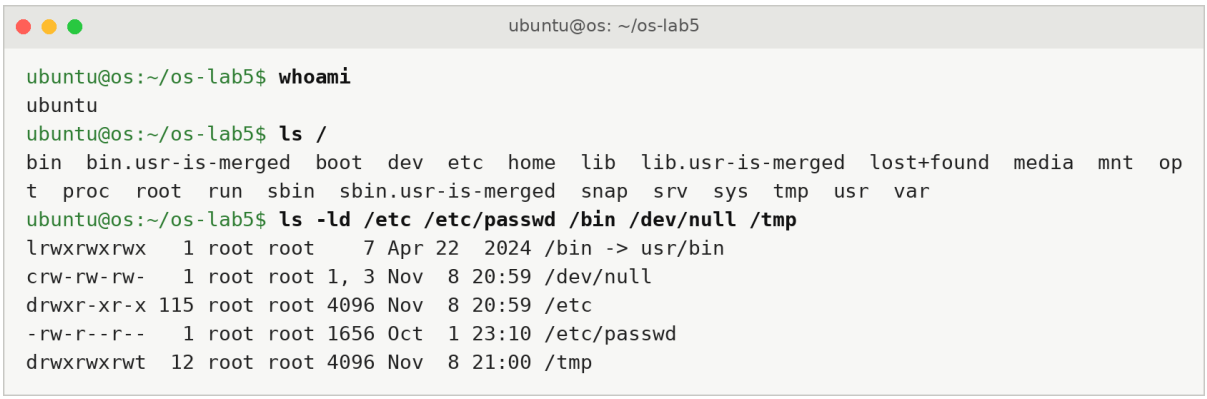


Figure 1.1 – The root directory and the types of five paths

Then the device files in /dev were listed and the real type of three paths was asked from the `file` command:

```
ls -l /dev | head
ls -l /dev/null /dev/sda
file /usr/bin/ls /etc/passwd /bin
```

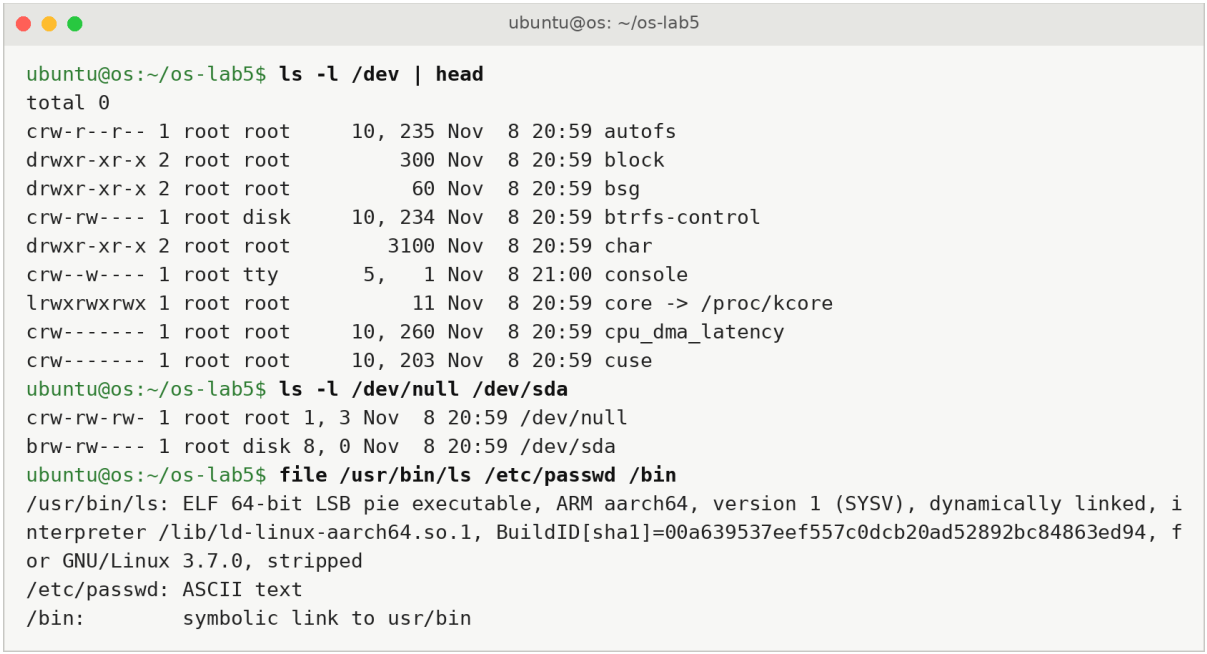


Figure 1.2 – Device files in /dev and the output of file

The last command of this part compares the size of two trees:

```
du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
```



Figure 1.3 – The sizes of /var/log and /usr

1.2. Observations

(1) The type is the first letter of the `ls -l` line. `/etc` is `d`, a directory. `/etc/passwd` is `-`, a regular file. `/bin` is `l`, a symbolic link. `/dev/null` is `c`, a character device. `/tmp` is `d`, a directory (the `t` at the end of its mode is a special bit, see Part 3).

(2) On this system `/bin` is not a directory but a symbolic link to `usr/bin` (`lrwxrwxrwx ... /bin -> usr/bin`, and `file` says `symbolic link to usr/bin`). The programs of the old `/bin` and of `/usr/bin` live in one place.

(3) `/dev/null` is a character device (`crw-rw-rw-`, numbers `1, 3`); `/dev/console` and `/dev/autofs` are character devices too. `/dev/sda` is a block device (`brw-rw----`, numbers `8, 0`, group `disk`): it is the whole virtual disk. For device files `ls -l` prints the major and minor number in the place where a regular file has its size.

(4) `/usr` is much bigger: 2.9 GB against 40 MB in `/var/log`, roughly 70 times. `/usr` holds the installed programs and libraries, `/var/log` only text logs.

Part 2. Inodes and Links

2.1. Work Done

A file with one line of text was created and its inode was read in two ways, with `stat` and with `ls -li`:

```
echo hello > notes.txt
stat notes.txt
ls -li notes.txt
```

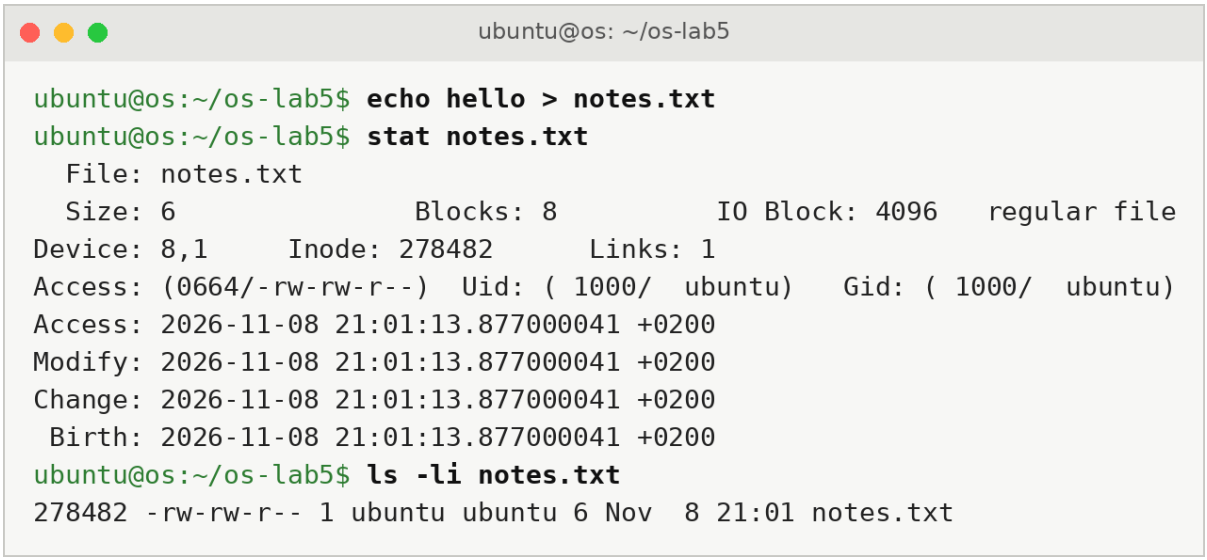


Figure 2.1 – stat and ls -li for notes.txt

A hard link was made, and then the original name was removed:

```
ln notes.txt hard.txt
ls -li notes.txt hard.txt
rm notes.txt
cat hard.txt ; ls -li hard.txt
```

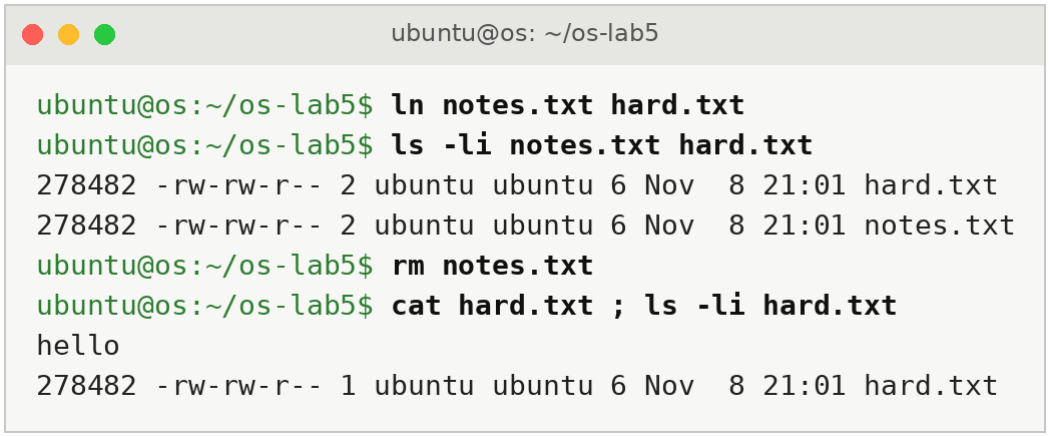


Figure 2.2 – A hard link, and the original name removed

The same was repeated with a symbolic link, removing its target this time:

```
ln -s hard.txt soft.txt
ls -li hard.txt soft.txt
cat soft.txt
rm hard.txt
cat soft.txt ; ls -l soft.txt
```

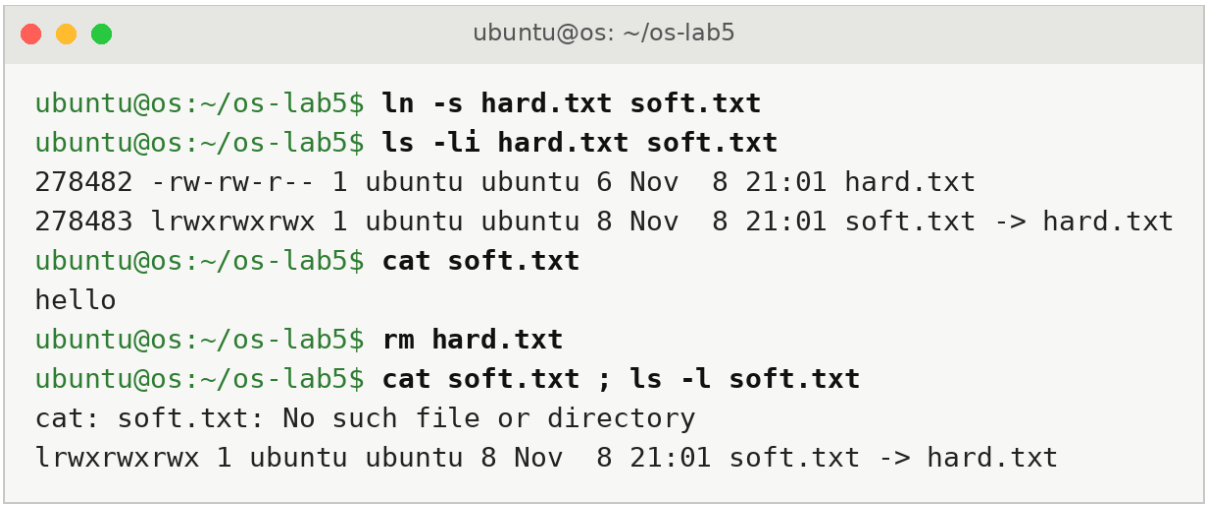


Figure 2.3 – A symbolic link, and its target removed

The link count of a directory was read before and after two subdirectories were created in it:

```
mkdir d ; ls -ld d
mkdir d/x d/y ; ls -ld d
```

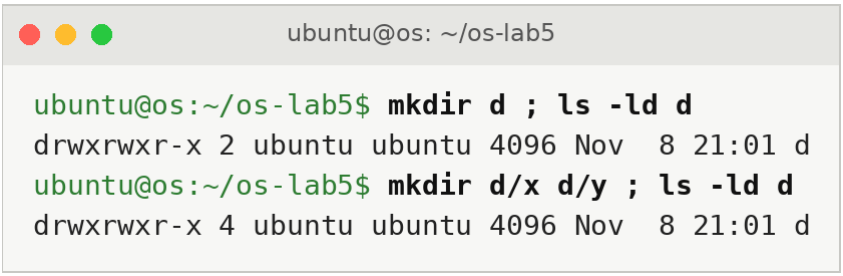


Figure 2.4 – The link count of a directory before and after two subdirectories

Finally the size of a file was compared with the space it occupies; the 1 GB file was created with `truncate` and removed at once:

```
echo x > one.txt ; ls -l one.txt ; du -h one.txt
truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
rm big.img
```

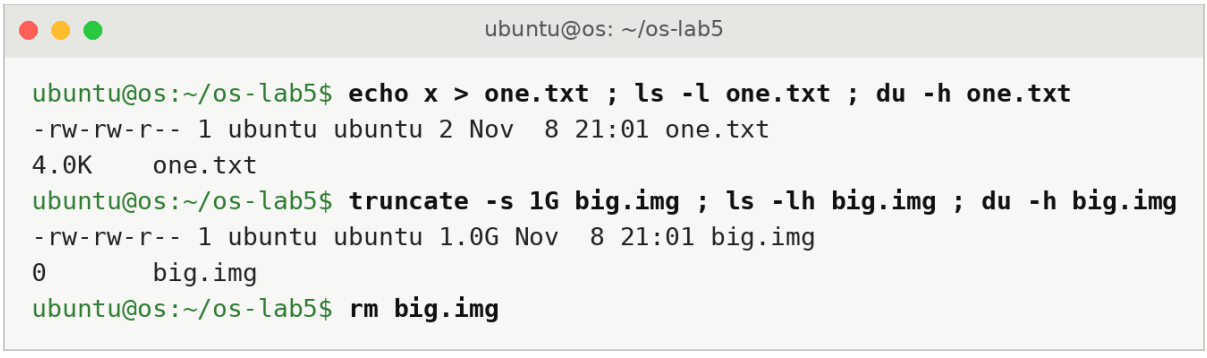


Figure 2.5 – File size in ls and occupied space in du

2.2. Observations

(1) In the `stat` output the line `Inode: 278482` is the inode number and `Links: 1` is the number of names the file has.

(2) `notes.txt` and `hard.txt` were two names of the same inode 278482, and after `ln` both lines showed link count 2 (Figure 2.2). `rm notes.txt` removed only one name from the directory and lowered the count to 1, so `hard.txt` still opened the same data and `cat` printed `hello`. The data blocks are freed only when the count reaches 0 and no process has the file open.

(3) A symbolic link is a separate file (own inode 278483, type `l`) whose content is a path, here the text `hard.txt`. After `rm hard.txt` nothing has that name any more, so opening the link fails with `No such file or directory`, although `soft.txt` itself still exists. Its size is 8 bytes because the stored path `hard.txt` has 8 characters.

(4) A new directory has 2 links: its name in the parent directory and its own entry `.`. Each of the two subdirectories `x` and `y` contains an entry `..` that points back to `d`, which adds two links, so 2 + 2 = 4 (Figure 2.4).

(5) The file system gives space in whole blocks; `stat` shows `IO Block: 4096` and `Blocks: 8` (8 units of 512 bytes), so the 2-byte file occupies one 4 KB block and `du` prints 4.0K. The 1 GB file made by `truncate` is a sparse file: its length is 1 GB, but no data was written and no block was allocated, so `ls` shows 1.0G and `du` shows 0.

Part 3. Permissions

3.1. Work Done

A file was closed with different modes; as its owner I could always change the mode back, but could read the file only when the read bit was set:

```
echo secret > s.txt
chmod 600 s.txt ; ls -l s.txt
chmod 000 s.txt ; cat s.txt
chmod u+r s.txt ; cat s.txt
```

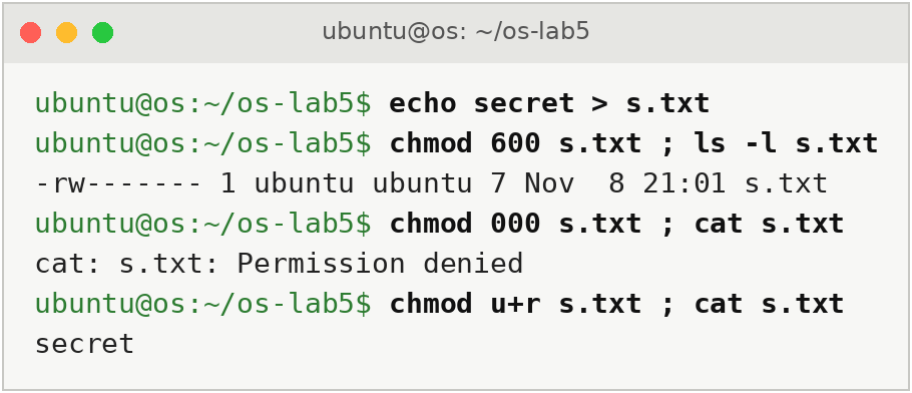


Figure 3.1 – A file with modes 600 and 000, then with the read bit restored

The umask and the permissions of a new file and a new directory:

```
umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
```



Figure 3.2 – The umask and the permissions of new objects

The directory experiment: a file `box/f` was created, then the directory was given first only the read bit and then only the execute bit:

```
mkdir box ; echo hi > box/f
chmod u=r,go= box ; ls box ; cat box/f
chmod u=x,go= box ; ls box ; cat box/f
chmod 755 box
```

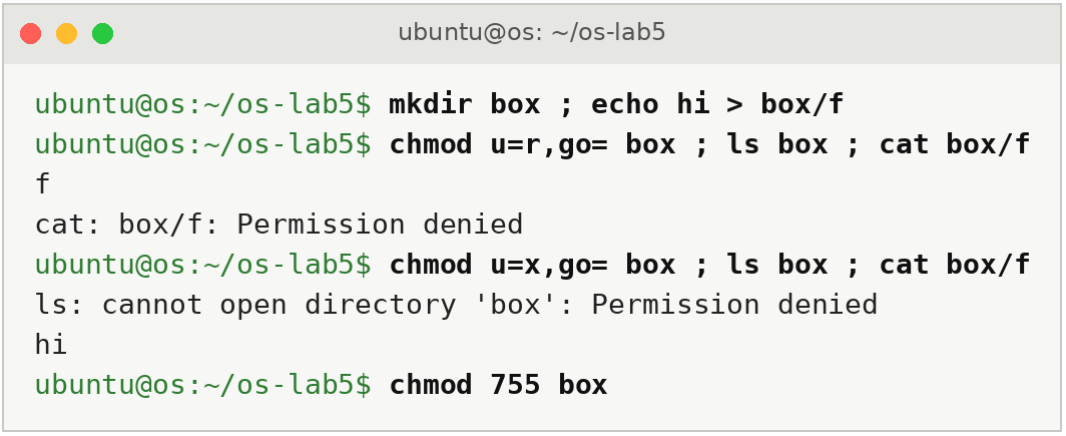


Figure 3.3 – A directory with only r, and with only x

Special bits and a file that belongs to another user:

```
ls -l /usr/bin/passwd ; ls -ld /tmp
cat /etc/shadow
ls -l /etc/shadow
```

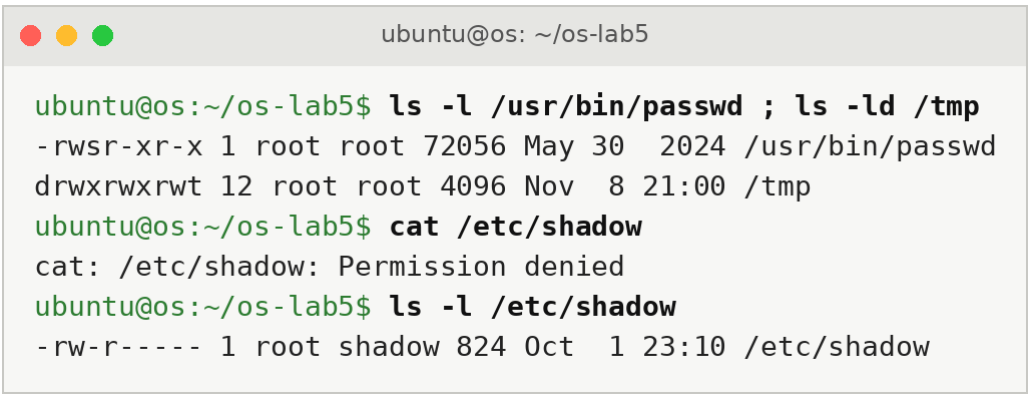


Figure 3.4 – The setuid bit of passwd, the sticky bit of /tmp and /etc/shadow

3.2. Observations

(1) The four common modes, with a typical use:

Mode	rwx string	Typical use
600	rw-------	a private file, for example a secret key or a token
644	rw-r--r--	an ordinary document or a configuration file that everybody may read
755	rwxr-xr-x	a program or a public directory that everybody may run or enter
700	rwx------	a private directory, for example `~/.ssh`

(2) With mode 000 the file has no permission bits for anybody. The owner is checked against the owner bits, which are empty, so reading fails (`Permission denied`, Figure 3.1). Ownership does not grant access; it only gives the right to change the mode. That is why `chmod u+r` worked and the next `cat` printed `secret`.

(3) The umask is `0002`. New files start from 666 and new directories from 777, and the umask removes the bits it contains (write for others). The result is 664 for `new.txt` (`-rw-rw-r--`) and 775 for `newdir` (`drwxrwxr-x`), exactly what Figure 3.2 shows.

(4) On a directory `r` allows reading the list of names and `x` allows passing through it to reach an entry by its name. With only `r`, `ls box` printed the name `f` but `cat box/f` was denied. With only `x`, `ls box` was denied, but `cat box/f` still printed `hi`, because the name was known.

(5) The `s` in `-rwsr-xr-x` is the setuid bit: the program runs with the rights of its owner, root, not of the user who started it. `passwd` needs it because it has to change `/etc/shadow`, a file that only root may write; for the user `ubuntu` even `cat /etc/shadow` fails (`-rw-r----- root shadow`). The `t` in `drwxrwxrwt` is the sticky bit: everybody may create files in `/tmp`, but only the owner of a file may delete or rename it.

Part 4. Disks and the Kernel as Files

4.1. Work Done

The disks, the free space and the file system type of the root were read with:

```
lsblk
df -h
mount | grep " / "
```

ubuntu@os: ~/os-lab5

```
ubuntu@os:~/os-lab5$ lsblk
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda       8:0    0   20G  0 disk
├─sda1    8:1    0   19G  0 part /
├─sda15   8:15   0   99M  0 part /boot/efi
└─sda16 259:0    0  923M  0 part /boot
vda     253:0    0   54K  1 disk
ubuntu@os:~/os-lab5$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           391M  1.1M  390M   1% /run
efivarfs        256K   15K  242K   6% /sys/firmware/efi/efivars
/dev/sda1        19G  3.4G   15G  19% /
tmpfs           2.0G     0  2.0G   0% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
tmpfs           2.0G     0  2.0G   0% /run/qemu
/dev/sda16      891M  114M  715M  14% /boot
/dev/sda15       98M  6.4M   92M   7% /boot/efi
tmpfs           391M   16K  391M   1% /run/user/1000
ubuntu@os:~/os-lab5$ mount | grep " / "
/dev/sda1 on / type ext4 (rw,relatime,discard,errors=remount-ro,commit=30)
```

Figure 4.1 – The disk, its partitions and the mounted file systems

The kernel data were read as ordinary files from `/proc`:

```
grep -c processor /proc/cpuinfo
head -3 /proc/meminfo
head -5 /proc/self/status
ls -l /proc/self/fd
```

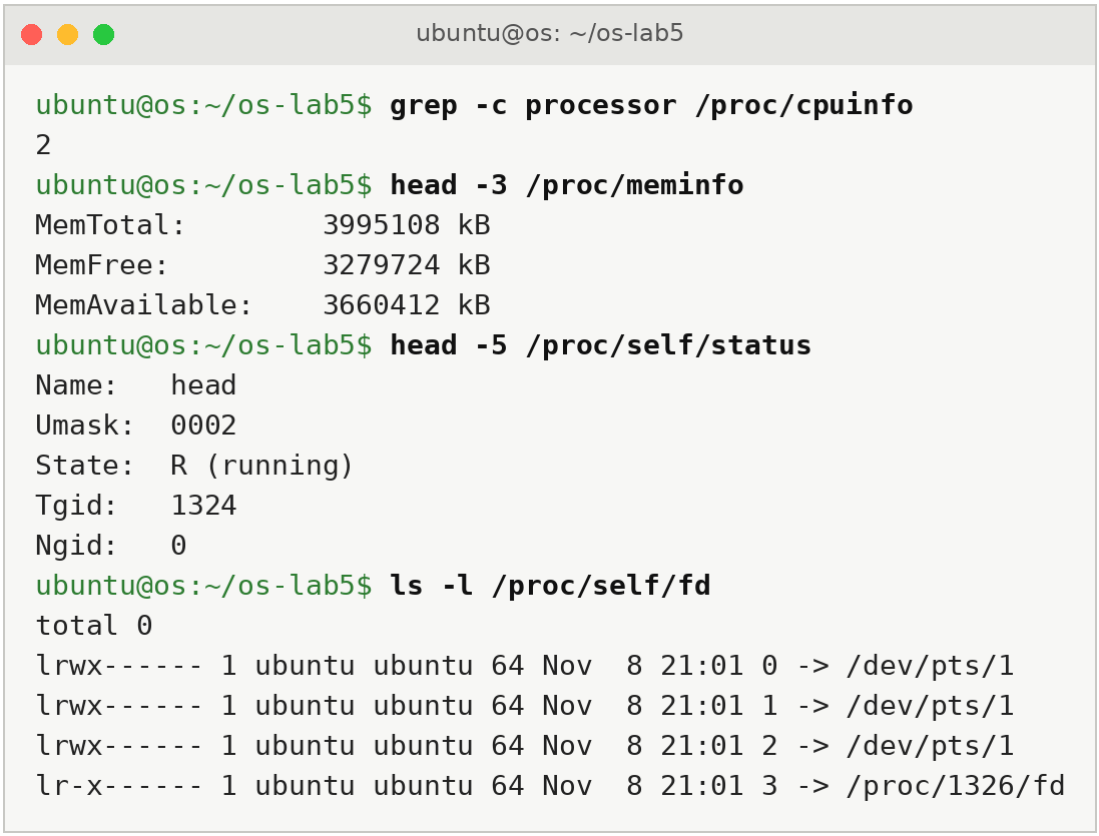


Figure 4.2 – CPU, memory and process data read from /proc

4.2. Observations

(1) The disk `sda` has 20 GB and three partitions. Partition `sda1` is mounted at `/` and has the file system `ext4` (`/dev/sda1 on / type ext4`):

```
sda       20G  disk
|-- sda1    19G  mounted at /       (ext4)
|-- sda15   99M  mounted at /boot/efi
`-- sda16  923M  mounted at /boot
```

(2) The root file system is 19 GB, of which 3.4 GB is used (19 %) and 15 GB is free.

(3) According to `/proc` the virtual machine has 2 CPU cores (`grep -c processor` printed 2) and `MemTotal: 3995108 kB`, about 3.8 GiB of RAM. These are the values of the virtual machine, not of the Mac.

(4) `/proc/self/status` showed `Name: head`. `/proc/self` always means the process that is reading, and the process that opened the file was `head`. In the same way `/proc/self/fd` showed the descriptors of `ls`: 0, 1 and 2 point to the terminal `/dev/pts/1`, and 3 is the directory `/proc/1326/fd` that `ls` itself was reading.

Part 5. Stretch Tasks

5.1. Magic Numbers

A file named `notes.txt` was filled with compressed data, and `file` was asked for its type:

```
gzip -c /etc/passwd > notes.txt ; file notes.txt
xxd notes.txt | head -1
```

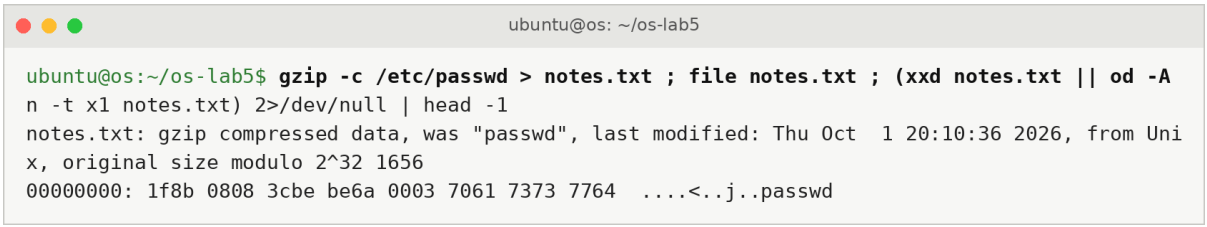


Figure 5.1 – file recognizes gzip data in a file named notes.txt

The name says text, but `file` answered `gzip compressed data`. It looks at the first bytes, not at the name: the dump starts with `1f 8b`, the magic number of gzip, followed by `08`, the deflate method. The same header contains the original name `passwd`.

5.2. Deleting a File That Is Still Open

A file was opened on descriptor 3, deleted, and the descriptors of the shell were listed; then the descriptor was closed:

```
truncate -s 1M bigfile
exec 3< bigfile ; rm bigfile ; ls -l /proc/$$/fd
exec 3<&- ; ls -l /proc/$$/fd
```

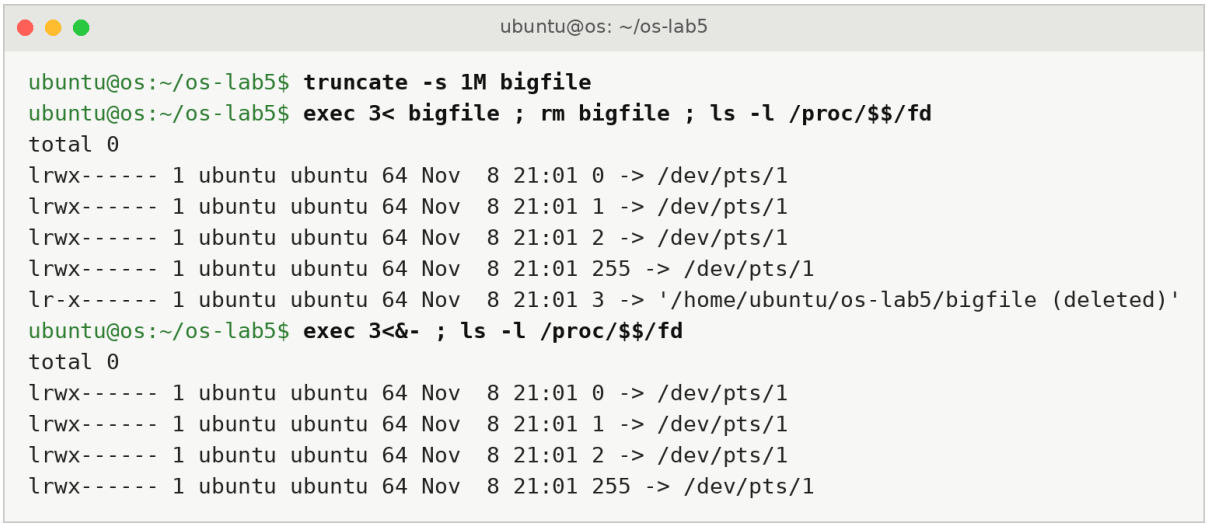


Figure 5.2 – A deleted file kept alive by an open descriptor

While the descriptor was open the link of descriptor 3 pointed to `/home/ubuntu/os-lab5/bigfile (deleted)`: the name was gone, but the inode was still in use, so the file system could not free it. After `exec 3<&-` the entry 3 disappeared from the list; this is the moment when the last reference is dropped and the space is really freed. (The file was created with `truncate`, so it occupied no real blocks, but the rule is the same for any file.)

Conclusions

In this laboratory work the Linux file system was checked on a running system. The tree contains regular files, directories, symbolic links and device files, and the first letter of `ls -l` tells them apart. A name is only an entry in a directory; the file itself is an inode with a link count. This explains why a file survives the removal of one of its names, why a symbolic link breaks when its target is removed, and why a directory has the link counts 2, 4 and so on.

The permission experiments showed that the owner is not special: mode 000 locked the owner out, and for a directory `r` and `x` mean different things. The special bits `s` and `t` solve concrete problems, a program that must write `/etc/shadow` and a shared directory like `/tmp`. The difference between `ls` and `du` showed that the size of a file and the space it uses are different numbers.

Finally, `/proc` showed that the kernel itself is presented as files: the CPU count, the memory and the descriptors of a process are read with the same tools as any text file.